A Comprehensive Measure of Well-Being

Project Description:

The Human Development Index (HDI) is a statistical composite index of life expectancy, education, and per capita income indicators, which are used to rank countries into four tiers Very High, High, Medium, and Low of human development. A country achieves a higher HDI when its population enjoys a longer lifespan, better educational attainment, and a higher Gross National Income (GNI PPP) per capita.

The HDI was developed to emphasize that people and their capabilities should be the primary measure of a country's development rather than economic growth alone. It provides a broader perspective on development by considering health, education, and living standards together. The index is widely used by governments, researchers, policymakers, and international organizations to evaluate progress, compare countries, and identify areas requiring improvement.

Scenarios

Scenario 1: Predicting Very High Human Development

A user selects a country with high life expectancy, strong mean years of schooling, expected years of schooling, and a high GNI per capita. Based on these indicators, the model predicts a Very High HDI score, classifying the country among the most developed nations. This helps users understand how strong performance across multiple development dimensions contributes to overall human development.

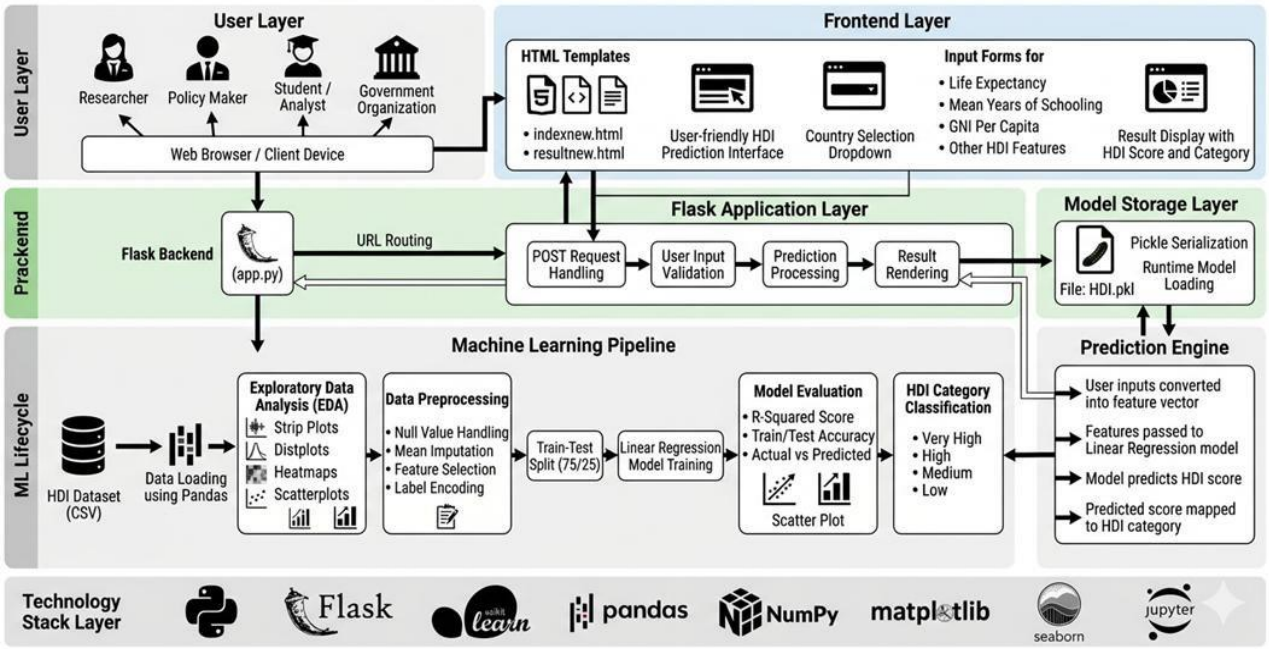
Scenario 2: Identifying Development Gaps in Emerging Economies

A policymaker inputs mid-range values representing a developing nation, including moderate life expectancy, average educational attainment, and a moderate income level. The model returns a Medium HDI score, providing insights into areas where improvements in healthcare, education, or income generation could significantly enhance human development outcomes.

Scenario 3: Assessing Countries Requiring Development Intervention

A researcher evaluates a country with relatively low life expectancy, limited educational opportunities, and low GNI per capita. The model predicts a Low HDI score and highlights the country's developmental challenges. Such predictions can support governments and development organizations in prioritizing investments and policy interventions.

Technical Architecture:



Description: The A Comprehensive Measure of Well-Being System is a web-based machine learning application developed using the Flask Web Application Framework to predict the overall well-being score based on user-provided parameters. The system receives user inputs through a simple web interface, where Flask handles the request and forwards the data to the Linear Regression Model for prediction, which is then classified into an appropriate Well-Being Category Finally, the system displays the prediction result, including both the predicted score and its corresponding category, through the web interface. This architecture provides a simple, reliable, and scalable framework by integrating user input, Flask-based backend processing, model loading, prediction, and result presentation into a seamless workflow for well-being assessment.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- Anaconda Environment Setup: https://www.anaconda.com/download
- PyCharm IDE: https://www.jetbrains.com/pycharm/
- NumPy Documentation: https://numpy.org/doc/stable/
- Pandas Documentation: https://pandas.pydata.org/docs/
- Scikit-learn Documentation: https://scikit-learn.org/stable/
- Matplotlib Documentation: https://matplotlib.org/stable/
- Seaborn Documentation: https://seaborn.pydata.org/
- Flask Documentation: https://flask.palletsprojects.com/

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Research and select the appropriate generative AI model from Groq for social media content generation.
- **Activity 1.2:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.3:** Set up the development environment, installing necessary libraries and dependencies for Flask and the Groq .

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Caption Generator, Hashtag Suggester, and Call-to-Action Generator.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our social media content generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating captions, hashtags, and CTAs across different platforms and tones.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select a best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the CaptionAI application, focusing on the types of content it will generate (captions, hashtags, and calls-to-action across Instagram and LinkedIn).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to social media copywriting tasks.
- **Select the Optimal Model:** Choose **llama-3.3-70b-versatile** for its balance of creative output quality and generation speed, set with a temperature of 0.8 and max_tokens of 2048 for rich, varied captions.

Activity 1.2: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, and Groq integration.
- **Detail Frontend Functionality:** Outline how users interact with the application entering product/campaign information, selecting a platform (Instagram or LinkedIn), and choosing a tone (Professional, Casual, or Promotional).
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using TONE_DESCRIPTIONS and PLATFORM_TIPS dictionaries, calls the Groq , parses the JSON response via extract_json(), and returns structured results to the frontend.

Activity 1.3: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

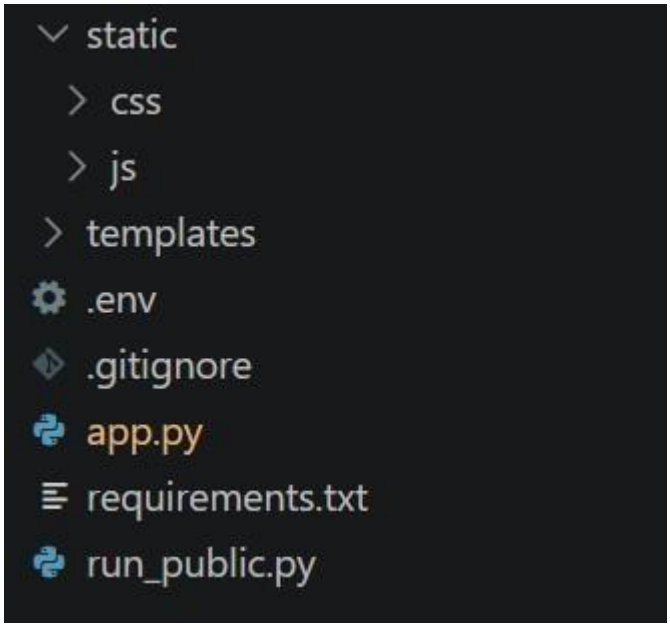
```
D:\projects>python -m venv myenv

D:\projects>"d:\projects\venv\Scripts\activate"
```

```
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features Captions, Hashtags, and CTAs as a unified AI response driven by a single structured prompt.

- **Caption Generator:** Generates three unique, platform-appropriate captions. For Instagram/Casual tone, includes relevant emojis and conversational language. For LinkedIn/Professional tone, produces insight-led, value-driven copy.
- **Hashtag Suggester:** Produces ten hashtags per request a curated mix of high-reach popular tags and targeted niche tags relevant to the product or campaign.
- **CTA Generator:** Returns three short (under 10 words), punchy, action-oriented call-to-action phrases calibrated to the selected platform and tone.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the home page and content generation endpoint.

Process User Input:

- Create a form in index.html for users to submit their product/campaign information, platform, and tone.
- Use Flask's request.get_json() to retrieve JSON data submitted from the frontend via AJAX.
- Validate that product_info is non-empty and within the 2000-character limit before processing.

Integrate API Calls:

- Within the /generate route handler, call the build_prompt() function to construct a structured prompt combining tone descriptions and platform-specific tips.
- Submit the prompt to the Groq API using client.chat.completions.create() with the llama-3.3-70b-versatile model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for CaptionAI's two endpoints, each serving a distinct purpose: ◦ **/** — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

◦ **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```
@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()
    product_info = data.get("product_info", "").strip()
    platform = data.get("platform", "instagram").lower()
    tone = data.get("tone", "professional").lower()
    if not product_info:
        return jsonify({"error": "Product information is required."}), 400
    if len(product_info) > 2000:
        return jsonify({"error": "Input too long. Please keep it under 2000 characters."}), 400
    try:
        prompt = build_prompt(product_info, platform, tone)
```

```
        chat_completion = client.chat.completions.create(
            messages=[
                {
                    "role": "system",
                    "content": "You are a social media content expert. Always respond with valid JSON only, no markdown formatting."
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            model="llama-3.3-70b-versatile",
            temperature=0.8,
            max_tokens=2048,
        )
        response_text = chat_completion.choices[0].message.content
        result = extract_json(response_text)
        # Validate structure
        if not all(k in result for k in ["captions", "hashtags", "ctas"]):
            raise ValueError("Invalid response structure from AI")
        return jsonify({
            "success": True,
            "captions": result["captions"],
            "hashtags": result["hashtags"],
            "ctas": result["ctas"],
            "platform": platform,
            "tone": tone
        })
    except ValueError as e:
        import traceback
        traceback.print_exc()
        return jsonify({"error": f"Failed to parse AI response: {str(e)}"}), 500
    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({"error": f"Generation failed: {str(e)}"}), 500
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```
TONE_DESCRIPTIONS = {
    "professional": "formal, authoritative, and business-oriented. Use clear, concise language that conveys expertise and credibility.",
    "casual": "friendly, conversational, and relatable. Use informal language, emojis, and a warm tone that feels personal.",
    "promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers to drive engagement."
}
```

- **PLATFORM_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).

```
PLATFORM_TIPS = {
    "instagram": "optimized for Instagram — visually descriptive, emotionally engaging, with strong visual storytelling. Max 2200 characters.",
    "linkedin": "optimized for LinkedIn — professional insights, value-driven, thought leadership style. Max 3000 characters."
}
```

- **build_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

```
def build_prompt(product_info: str, platform: str, tone: str) -> str:
    tone_desc = TONE_DESCRIPTIONS.get(tone, "engaging")
    platform_tip = PLATFORM_TIPS.get(platform, "social media")

    return f"""You are an expert social media content strategist and copywriter.
```

Integrate the Groq AI Responses in Each Function:

- Call client.chat.completions.create() with a system message enforcing JSON-only responses and a user message containing the assembled prompt.

- **extract_json()**: a robust parser that first attempts direct json.loads(), then strips markdown code fences (```json blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.

```python
def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    # Try direct parse first
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    # Try extracting from markdown code block
    match = re.search(r'```(?:json)?\s*([\s\S]*?)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r'\{[\s\S]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains all three required keys — captions, hashtags, and ctas — before returning it to the frontend.
- **Home route:** / → renders index.html/generate (POST) → accepts JSON, returns {success,captions, hashtags, ctas, platform, tone}

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the CaptionAI brand logo and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.

- Integrate Font Awesome icons for platform indicators and result section headings, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark glassmorphism aesthetic with animated background blobs (blob-1, blob-2, blob-3) for visual depth.
- Use flexbox and grid layouts to organize the results into a two-column structure (captions on the left, hashtags and CTAs on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Caption Cards**: individual cards per caption, each with a one-click copy-to-clipboard button that provides visual feedback (checkmark + green highlight for 2 seconds).
- **Hashtag Chips**: rendered as pill-shaped chip elements inside a glass card, with automatic # prefix validation.
- **CTA List**: displayed as a clean unordered list inside a separate glass card with a bullhorn icon header.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

Render Results Dynamically:

- Implement the renderResults(data) function in main.js, which dynamically creates and injects DOM elements for captions, hashtags, and CTAs into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollIntoView().

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

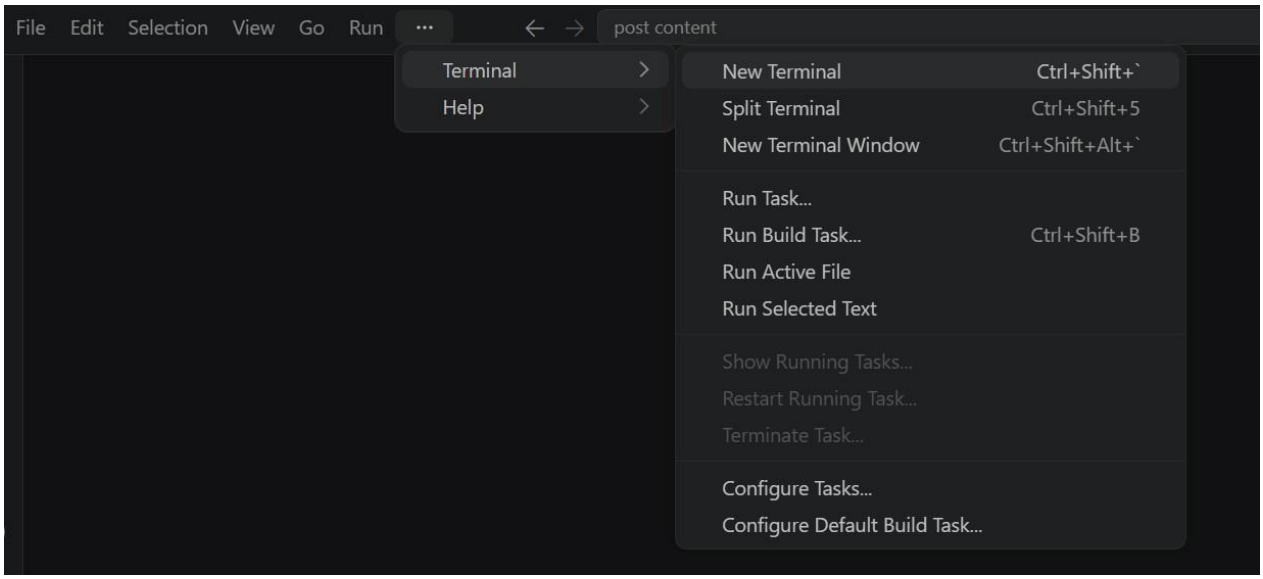
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

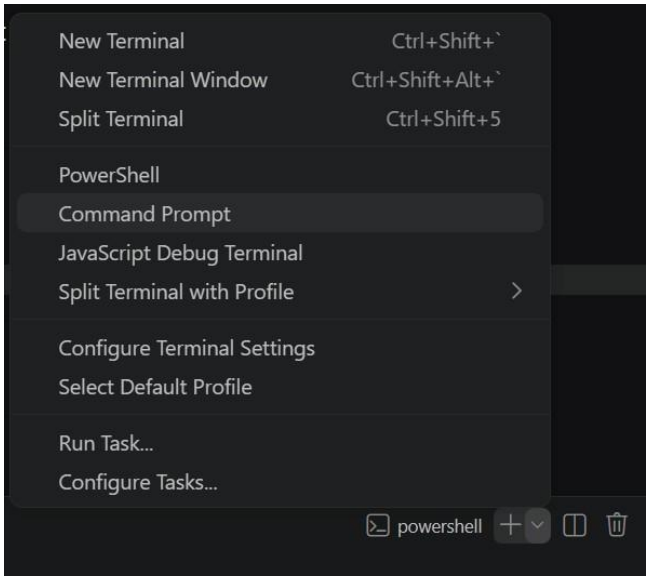
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv

D:\projects>"d:\projects\venv\Scripts\activate"

(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup: GROQ_API_KEY=your_groq_api_key_here

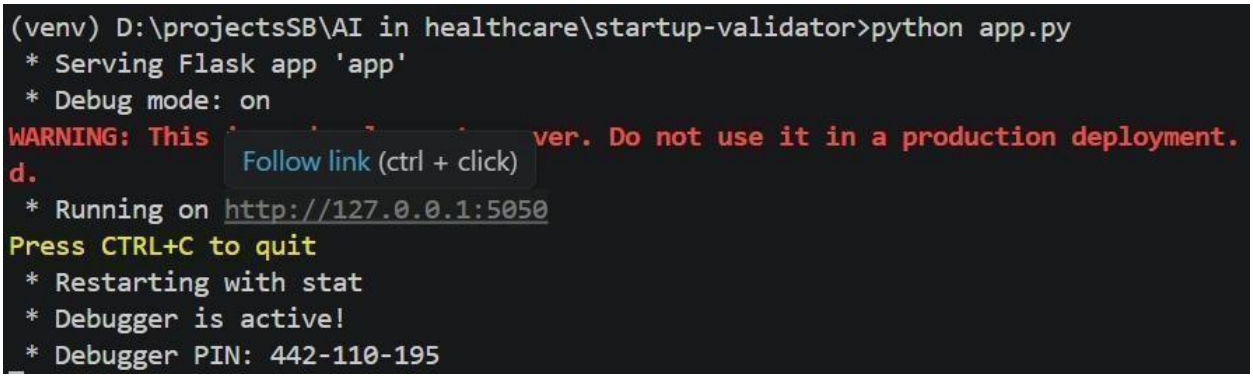
Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.



- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

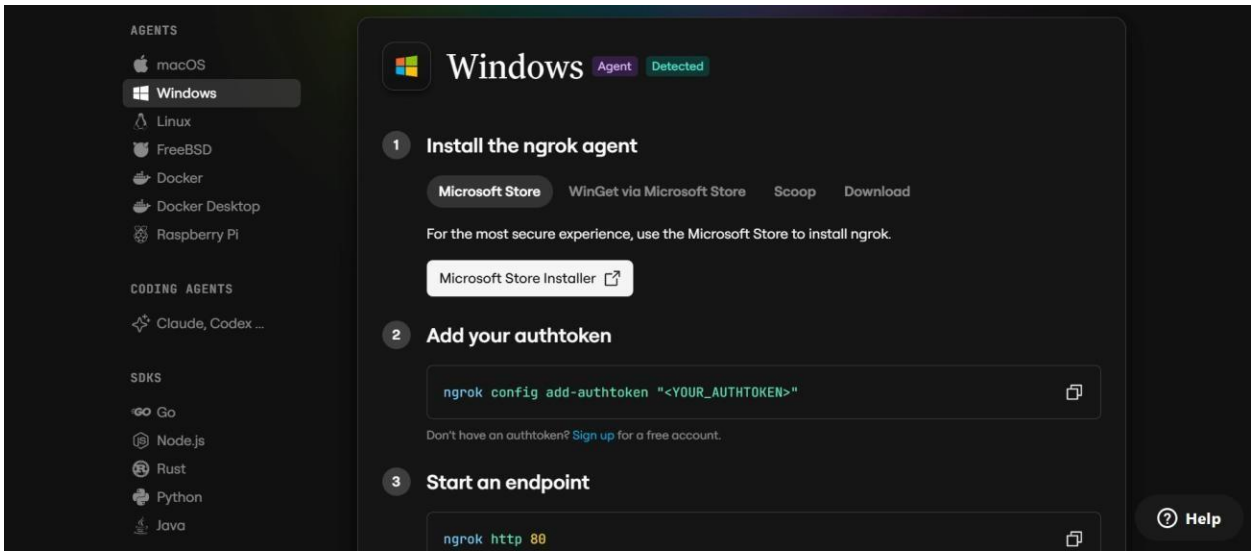
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on "Microsoft Store Installer".



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

  ========== YOUR PUBLIC WEBSITE URL IS LIVE!
  URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
  ==========

- Share this URL with anyone who can access your CaptionAI app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.

The **Human Development Index (HDI) Prediction System** is a machine learning-based web application developed to predict the Human Development Index category using important socio-economic indicators. The Human Development Index is a globally recognized measure used to evaluate the overall development of a country by considering factors such as life expectancy, education, and income. This project aims to simplify the prediction process by allowing users to enter the required input values through a user-friendly web interface and receive an instant prediction generated by a trained machine learning model.

The application is developed using the **Flask** web framework, which serves as the backend and connects the frontend with the machine learning model. The model is trained using historical HDI data after performing data preprocessing, feature selection, and model training. Once the training process is completed, the model is saved using **Joblib** and integrated into the Flask application. Whenever a user submits input values, the application processes the data, loads the trained model, predicts the HDI category, and displays the result on the webpage in real time.

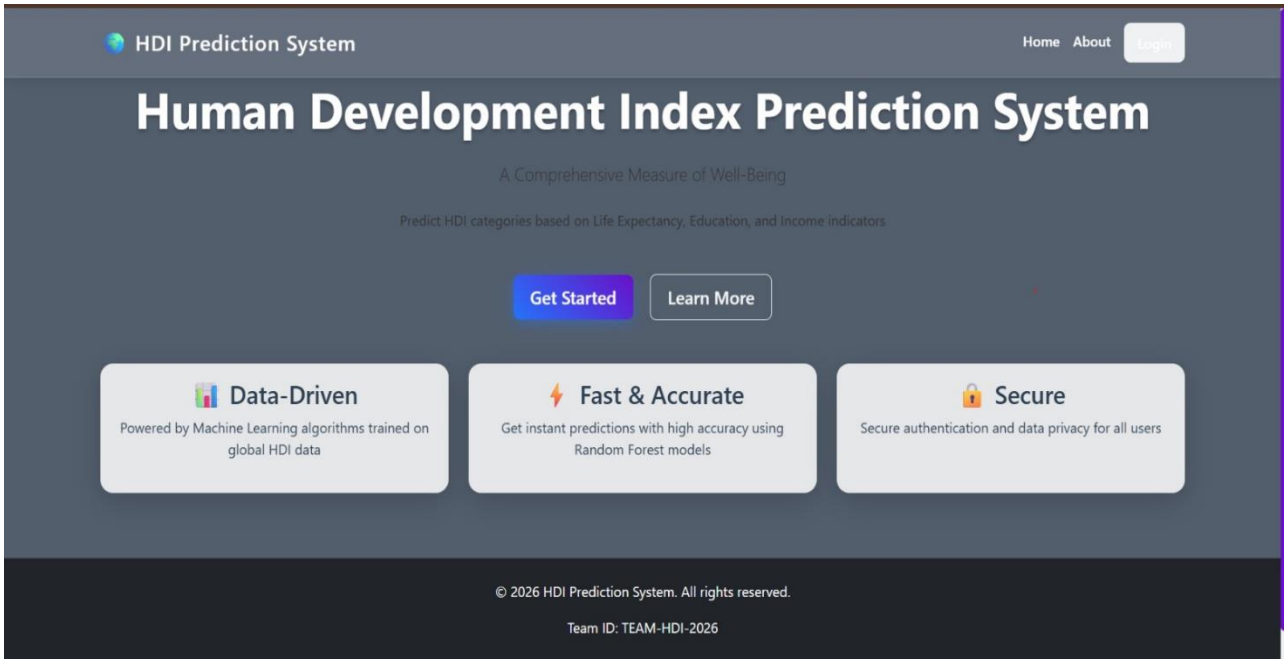
The project demonstrates the practical implementation of machine learning in solving real-world problems related to socio-economic development. It provides users with a simple, efficient, and interactive platform for understanding how different development indicators influence a country's Human Development Index. The application is designed with an intuitive interface using HTML and CSS, ensuring that users can easily enter data and understand the prediction results without requiring any technical knowledge.

To make the application accessible from anywhere, the project is deployed on the **Render** cloud platform. The project files are stored in a GitHub repository, which is connected to Render for deployment. During deployment, all required Python libraries are installed using the `requirements.txt` file, and the Flask application is executed using **Gunicorn** as the production web server. Once the deployment process is completed, Render generates a live URL through which users can access the application using any web browser without installing additional software.

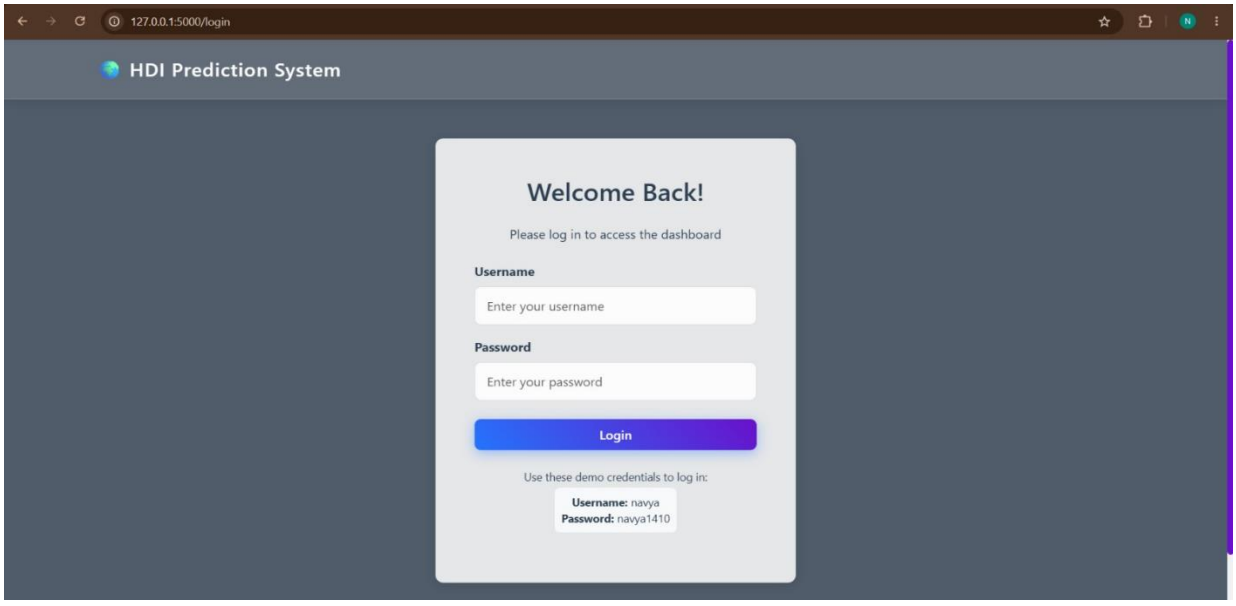
Overall, the HDI Prediction System demonstrates the complete machine learning workflow, including data collection, preprocessing, model training, model integration, web application development, and cloud deployment. The project not only provides accurate and real-time HDI predictions but also serves as an educational example of how machine learning models can be integrated with web technologies to build practical and scalable applications. With future enhancements such as larger datasets, advanced machine learning algorithms, interactive visualizations, and analytical dashboards, the system can be further improved to provide more comprehensive insights into human development and support better decision-making.

Exploring the Website's Web Pages:

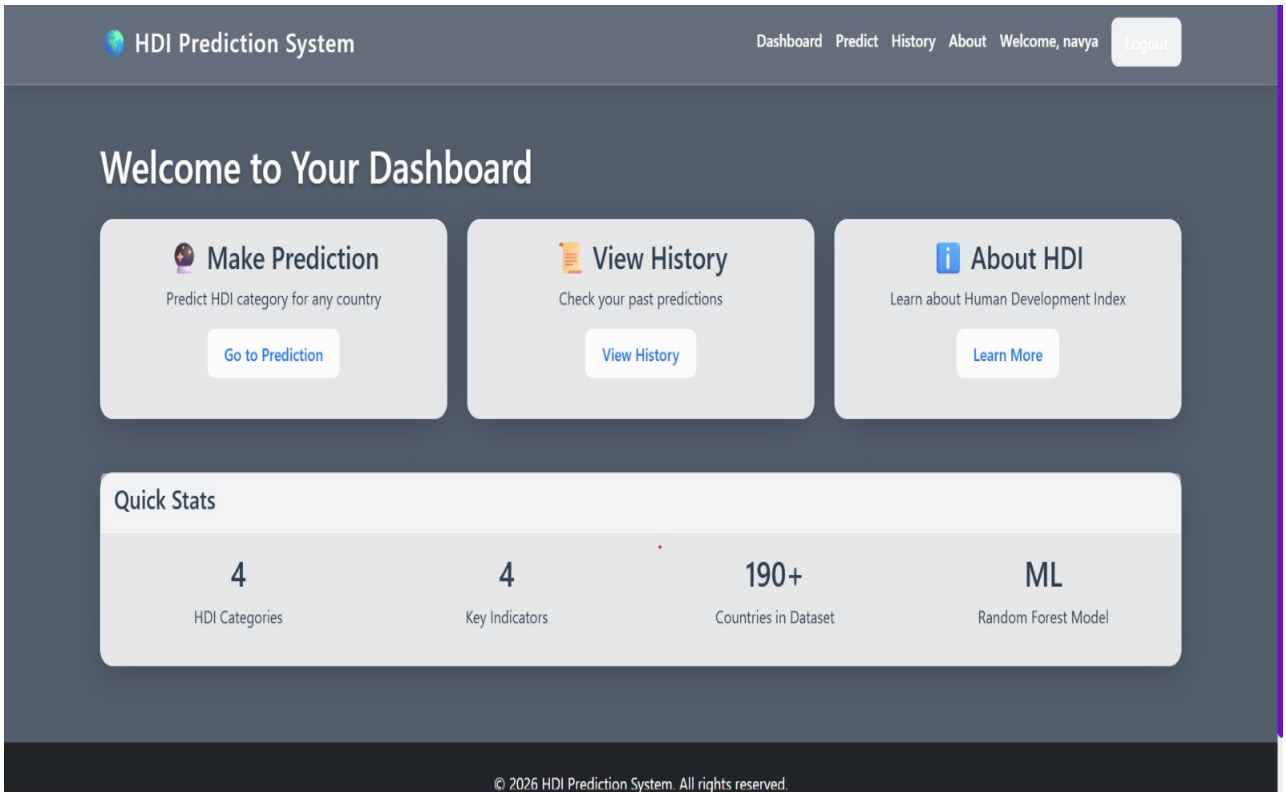
Home Page:



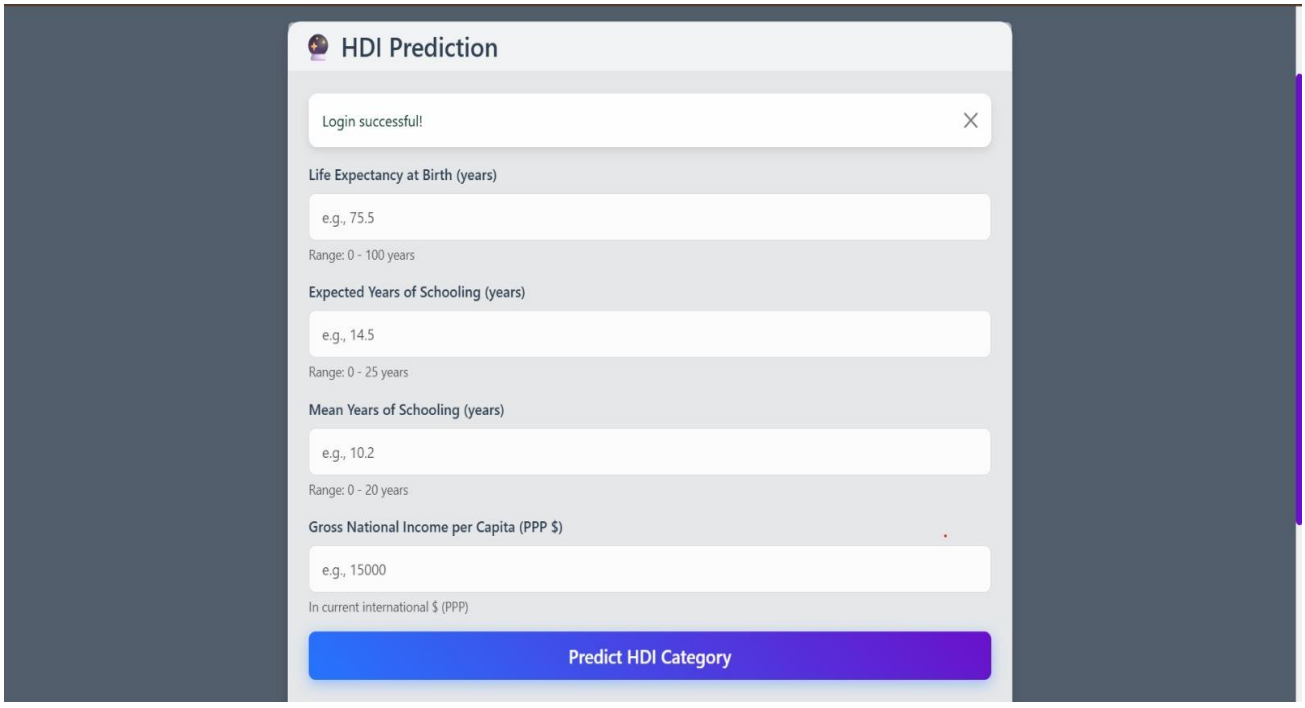
Login Page:



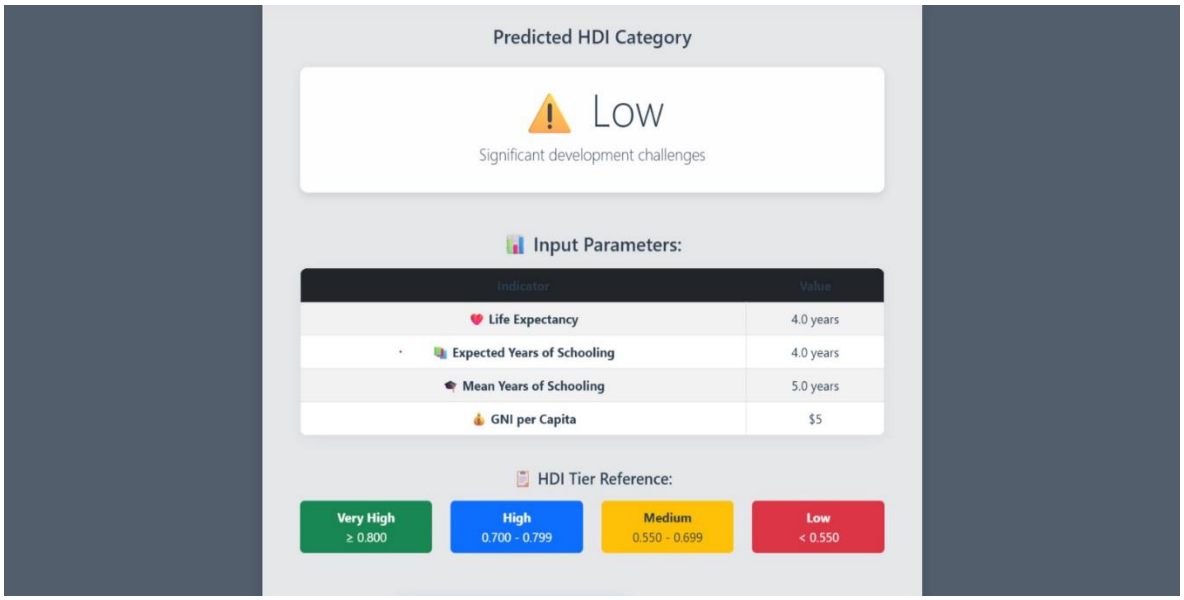
Dashboard Page:



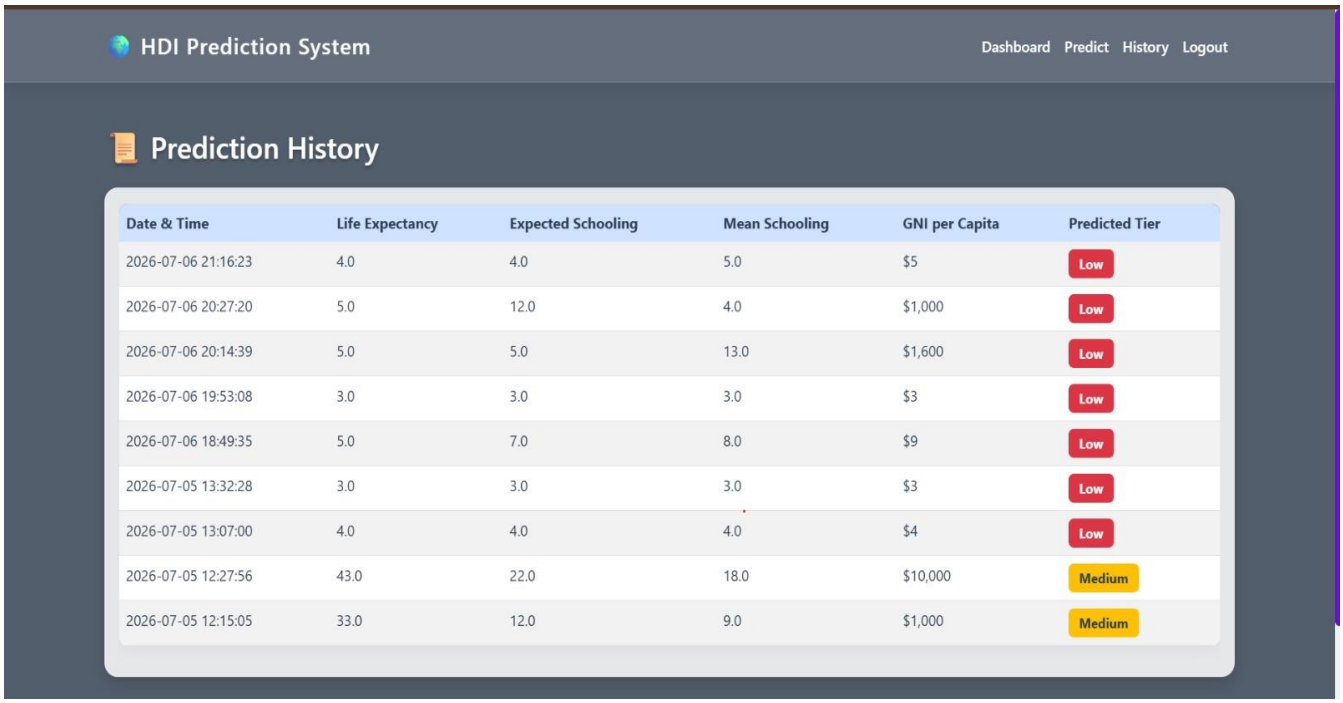
Prediction Page:



Result Page:

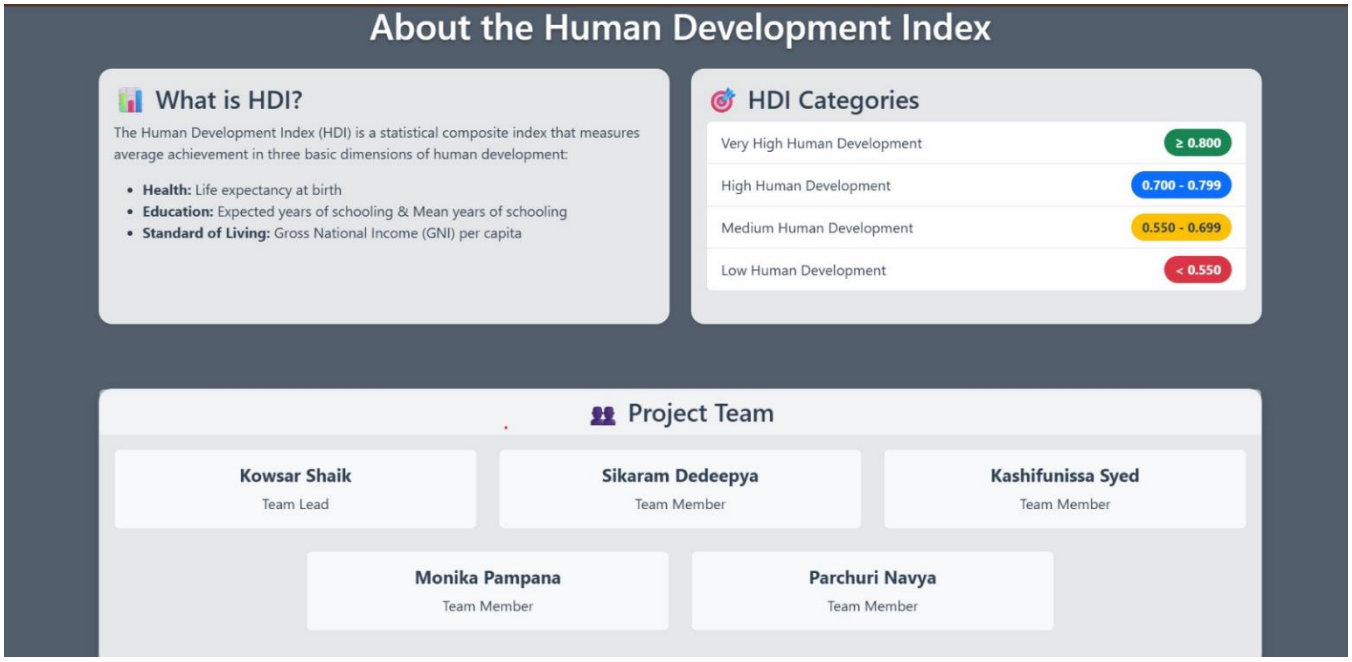


History:



Date & Time	Life Expectancy	Expected Schooling	Mean Schooling	GNI per Capita	Predicted Tier
2026-07-06 21:16:23	4.0	4.0	5.0	$5	Low
2026-07-06 20:27:20	5.0	12.0	4.0	$1,000	Low
2026-07-06 20:14:39	5.0	5.0	13.0	$1,600	Low
2026-07-06 19:53:08	3.0	3.0	3.0	$3	Low
2026-07-06 18:49:35	5.0	7.0	8.0	$9	Low
2026-07-05 13:32:28	3.0	3.0	3.0	$3	Low
2026-07-05 13:07:00	4.0	4.0	4.0	$4	Low
2026-07-05 12:27:56	43.0	22.0	18.0	$10,000	Medium
2026-07-05 12:15:05	33.0	12.0	9.0	$1,000	Medium

About Page:



Dataset

Official Human Development Report (UNDP)

Input Features

- Life Expectancy
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) Per Capita

Output

HDI Categories:

- Very High
- High
- Medium
- Low

Description:

The HDI Prediction System is a machine learning web application that predicts a country's Human Development Index category using a Random Forest model with 90.47% accuracy. It analyzes four key indicators: Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and GNI Per Capita to classify countries into four categories: Very High, High, Medium, or Low human development. The system features an interactive dashboard, prediction history tracking, and secure login, using official UNDP data to help researchers and policymakers assess development levels quickly and accurately

Conclusion:

The Human Development Index (HDI) represents a paradigm shift from traditional economic metrics like GDP to a more holistic assessment of human well-being. By integrating **life expectancy, education, and income** into a single composite index, the HDI captures the multidimensional nature of development and recognizes that true progress extends beyond economic growth alone.

The HDI Prediction System demonstrates that this comprehensive approach is both measurable and actionable. With **90.47% accuracy**, the machine learning model validates that these three fundamental dimensions—**health (life expectancy), knowledge (schooling years), and standard of living (GNI per capita)**—serve as reliable indicators of a nation's overall human development status.